

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



LAB 2

KHAI THÁC TẬP PHỔ BIẾN BẰNG THUẬT TOÁN PREPOST

Nhóm Group 06

Mã số sinh viên Họ và tên

22120187 TRẦN THIÊN LỘC

23120095 LƯU ĐỨC TOÀN

23120245 NGUYỄN QUANG DUY

23120258 LƯU TRỌNG HIẾU

23120260 VĂN ĐÌNH HIẾU

Giảng viên hướng dẫn GS.TS Lê Hoài Bắc
Th.S Lê Nhật Nam

Môn học Khai thác dữ liệu và ứng dụng

Thành phố Hồ Chí Minh – 2026

Lời cảm ơn

Nhóm chúng em xin gửi lời cảm ơn chân thành đến quý thầy cô phụ trách học phần *Khai thác dữ liệu và ứng dụng* đã hướng dẫn, cung cấp kiến thức nền tảng và định hướng thực hiện đồ án.

Trong quá trình thực hiện, nhóm đã có cơ hội tìm hiểu sâu hơn về bài toán khai thác tập phổ biến, các cấu trúc dữ liệu đặc trưng và phương pháp đánh giá hiệu năng thuật toán trên các tập dữ liệu benchmark. Những kiến thức này giúp nhóm hiểu rõ hơn vai trò của Frequent Itemset Mining trong khai phá dữ liệu và các ứng dụng thực tế như phân tích giỏ hàng.

Do giới hạn về thời gian và kinh nghiệm, báo cáo có thể còn thiếu sót. Nhóm mong nhận được góp ý từ quý thầy cô để hoàn thiện hơn.

Tóm tắt

Khai thác tập phổ biến là một bài toán nền tảng trong khai phá dữ liệu, được sử dụng rộng rãi trong sinh luật kết hợp, phân tích giỏ hàng, phát hiện mẫu trong log hệ thống và nhiều lĩnh vực khác. Đề án này tập trung nghiên cứu và cài đặt thuật toán **PrePost**, một thuật toán khai thác tập phổ biến dựa trên cấu trúc **PPC-tree** và **N-list**.

Khác với các thuật toán sinh ứng viên như Apriori, PrePost mã hóa quan hệ tổ tiên – hậu duệ giữa các node trên cây bằng cặp thứ tự duyệt trước và duyệt sau. Từ đó, thuật toán có thể tính support của các itemset thông qua thao tác trên N-list mà không cần quét lại toàn bộ cơ sở dữ liệu giao dịch nhiều lần.

Trong đề án, nhóm trình bày cơ sở lý thuyết của bài toán Frequent Itemset Mining, phân tích chi tiết thuật toán PrePost, minh họa thuật toán bằng ví dụ thủ công, cài đặt thuật toán bằng ngôn ngữ Julia, kiểm tra độ đúng bằng cách so sánh với SPMF và đánh giá hiệu năng trên các tập dữ liệu benchmark. Ngoài ra, nhóm áp dụng kết quả frequent itemset vào bài toán phân tích giỏ hàng thông qua việc sinh luật kết hợp.

Từ khóa: Frequent Itemset Mining, PrePost, PPC-tree, N-list, association rules, market basket analysis.

Danh mục từ viết tắt

FIM	Frequent Itemset Mining – Khai thác tập phổ biến.
FI	Frequent Itemset – Tập phổ biến.
CFI	Closed Frequent Itemset – Tập phổ biến đóng.
MFI	Maximal Frequent Itemset – Tập phổ biến tối đại.
TDB	Transaction Database – Cơ sở dữ liệu giao dịch.
TID	Transaction Identifier – Mã định danh giao dịch.
minsup	Minimum Support – Ngưỡng hỗ trợ tối thiểu.
sup	Support – Độ hỗ trợ của một itemset.
conf	Confidence – Độ tin cậy của một luật kết hợp.
lift	Lift – Độ đo mức độ phụ thuộc giữa hai vế của luật kết hợp.
PPC-tree	Pre-order Post-order Coding Tree – Cây mã hóa theo thứ tự duyệt trước và duyệt sau.
pre	Pre-order code – Mã thứ tự duyệt trước của một node trong PPC-tree.
post	Post-order code – Mã thứ tự duyệt sau của một node trong PPC-tree.
N-list	Node-list – Danh sách mã hóa các node dùng để tính support trong PrePost.
SPMF	Sequential Pattern Mining Framework – Bộ công cụ tham chiếu cho khai phá mẫu.
CLI	Command Line Interface – Giao diện dòng lệnh.
RAM	Random Access Memory – Bộ nhớ truy cập ngẫu nhiên.
CSV	Comma-Separated Values – Định dạng dữ liệu bảng phân tách bằng dấu phẩy.

Danh sách hình vẽ

1	Minh họa cấu trúc PPC-tree với mã <i>pre</i> và <i>post</i>	4
2	PPC-tree của ví dụ cơ sở trước khi gán mã <i>pre</i> và <i>post</i>	10
3	PPC-tree của ví dụ cơ sở sau khi gán mã <i>pre</i> và <i>post</i>	11
4	PPC-tree một nhánh trong ví dụ đặc biệt	16
5	Pipeline cài đặt thuật toán PrePost	22
6	Giải mã khai thác đệ quy bằng N-list trong phiên bản tối ưu	23

Danh sách bảng

2	TDB của ví dụ cơ sở	8
3	Support của các item đơn trong ví dụ cơ sở	9
4	TDB sau khi sắp xếp theo thứ tự toàn cục	9
5	Các node của PPC-tree sau khi gán mã <i>pre</i> và <i>post</i>	10
6	N-list của các item đơn trong ví dụ cơ sở	11
7	Các frequent 2-itemset trong ví dụ cơ sở	12
8	Các frequent 3-itemset trong ví dụ cơ sở	13
9	Tập FI cuối cùng của ví dụ cơ sở	13
10	Kiểm tra chéo support của toàn bộ 2-itemset	14
11	Kiểm tra chéo support của toàn bộ 3-itemset	14
12	TDB của ví dụ đặc biệt	15
13	Support của các item đơn trong ví dụ đặc biệt	15
14	TDB sau khi sắp xếp trong ví dụ đặc biệt	15
15	Mã <i>pre</i> và <i>post</i> trong PPC-tree một nhánh	16
16	N-list của các item đơn trong ví dụ đặc biệt	16
17	Tập FI cuối cùng của ví dụ đặc biệt	17
18	Môi trường và công cụ cài đặt	19
19	Tổ chức các file mã nguồn chính	19
20	Năm CSDL toy dùng cho kiểm thử Level 2	26
21	Kết quả kiểm tra thuật toán trên năm CSDL toy	26
22	Kết quả so sánh phiên bản cơ bản và phiên bản tối ưu	27
23	Tổng hợp mức độ đáp ứng các yêu cầu cài đặt	28

Mục lục

Lời cảm ơn	i
Tóm tắt	ii
Danh mục từ viết tắt	iii
Danh mục hình vẽ	iv
Danh mục bảng biểu	v
1 Nền tảng lý thuyết	1
1.1 Bài toán khai thác tập phổ biến	1
1.2 Tính chất Apriori	2
1.3 Tổng quan thuật toán PrePost	3
1.4 Cấu trúc dữ liệu PPC-tree	3
1.5 Cấu trúc dữ liệu N-list	4
1.6 Giải mã thuật toán PrePost	5
1.7 Phân tích độ phức tạp	6
1.8 So sánh lịch sử với các thuật toán FIM khác	7
1.9 Kết luận chương	8
2 Ví dụ minh họa tay thuật toán PrePost	8
2.1 Ví dụ 1: Cơ sở	8
2.1.1 Cơ sở dữ liệu giao dịch ban đầu	8
2.1.2 Bước 1: Tính support của các item đơn	8
2.1.3 Bước 2: Sắp xếp lại các giao dịch	9
2.1.4 Bước 3: Xây dựng PPC-tree	9
2.1.5 Bước 4: Gán mã pre và post	10
2.1.6 Bước 5: Tạo N-list cho các item đơn	11
2.1.7 Bước 6: Sinh frequent 2-itemset bằng N-list	11
2.1.8 Bước 7: Sinh frequent 3-itemset	12
2.1.9 Tập kết quả cuối cùng	13
2.1.10 Kiểm tra chéo bằng liệt kê thủ công	13
2.2 Ví dụ 2: Tình huống đặc biệt PPC-tree một nhánh	14
2.2.1 Cơ sở dữ liệu giao dịch ban đầu	14
2.2.2 Bước 1: Tính support và thứ tự toàn cục	15
2.2.3 Bước 2: Xây dựng PPC-tree một nhánh	16
2.2.4 Bước 3: Gán mã pre và post	16
2.2.5 Bước 4: Tạo N-list	16

2.2.6	Bước 5: Khai thác các FI	16
2.2.7	Phân tích ý nghĩa của trường hợp đặc biệt	17
2.3	Kết luận chương	18
3	Cài đặt thuật toán PrePost	18
3.1	Môi trường và công cụ	18
3.2	Tổ chức mã nguồn	19
3.3	Biểu diễn dữ liệu và cấu trúc chính	20
3.3.1	Cơ sở dữ liệu giao dịch	20
3.3.2	PPC-tree	20
3.3.3	N-list	21
3.4	Quy trình cài đặt thuật toán PrePost	22
3.5	Phiên bản cơ bản và phiên bản tối ưu	22
3.6	Xử lý đầu vào, đầu ra và giao diện dòng lệnh	24
3.7	Kiểm thử Level 1 và Level 2	25
3.7.1	Mục tiêu kiểm thử	25
3.7.2	Bộ dữ liệu kiểm thử Level 2	25
3.8	Kiểm tra Level 3: so sánh phiên bản cơ bản và tối ưu	27
3.9	Đáp ứng các mức yêu cầu cài đặt	27
3.10	Nhận xét về cài đặt	28
3.11	Kết luận chương	29
4	Thực nghiệm và đánh giá	29
5	Ứng dụng thực tế	29
	Tài liệu tham khảo	30
A	Phụ lục	31

1 Nền tảng lý thuyết

1.1 Bài toán khai thác tập phổ biến

FIM là bài toán tìm tất cả các tập mục xuất hiện thường xuyên trong một TDB. Đây là bài toán nền tảng của khai phá luật kết hợp và nhiều ứng dụng khai phá mẫu khác như phân tích giỏ hàng, phân tích log hệ thống và phát hiện mẫu trong dữ liệu sinh học.

Cho tập tất cả các mục là:

$$I = \{i_1, i_2, \dots, i_m\}. \quad (1)$$

Một itemset là một tập con của I . Một TDB được ký hiệu là:

$$D = \{T_1, T_2, \dots, T_n\}, \quad (2)$$

trong đó mỗi giao dịch $T_j \subseteq I$ là một tập các mục xuất hiện cùng nhau trong một lần quan sát. Mỗi giao dịch thường được gán với một TID.

Với một itemset $X \subseteq I$, giao dịch T_j chứa X nếu và chỉ nếu:

$$X \subseteq T_j. \quad (3)$$

Độ hỗ trợ tuyệt đối của X trên TDB D được định nghĩa là số lượng giao dịch chứa X :

$$sup_{abs}(X) = |\{T_j \in D \mid X \subseteq T_j\}|. \quad (4)$$

Độ hỗ trợ tương đối của X là tỉ lệ giao dịch chứa X trên toàn bộ TDB:

$$sup_{rel}(X) = \frac{sup_{abs}(X)}{|D|}. \quad (5)$$

Trong báo cáo này, nếu không nói rõ, ký hiệu $sup(X)$ được hiểu là độ hỗ trợ tuyệt đối.

Cho một ngưỡng hỗ trợ tối thiểu $minsup$. Một itemset X được gọi là FI nếu:

$$sup(X) \geq minsup. \quad (6)$$

Bài toán FIM có thể được phát biểu hình thức như sau: với TDB D và ngưỡng $minsup$, tìm toàn bộ tập:

$$FI(D, minsup) = \{X \subseteq I \mid sup(X) \geq minsup\}. \quad (7)$$

Ngoài FI thông thường, hai khái niệm quan trọng khác là CFI và MFI. Một FI X được gọi là CFI nếu không tồn tại FI Y sao cho:

$$X \subset Y \quad \text{và} \quad \text{sup}(X) = \text{sup}(Y). \quad (8)$$

Nói cách khác, X là CFI nếu không thể thêm mục nào vào X mà vẫn giữ nguyên support.

Một FI X được gọi là MFI nếu không tồn tại FI Y sao cho:

$$X \subset Y. \quad (9)$$

MFI là những FI không có tập cha nào cũng frequent. Quan hệ giữa ba loại tập có thể viết như sau:

$$MFI \subseteq CFI \subseteq FI. \quad (10)$$

MFI giúp biểu diễn kết quả ngắn gọn hơn nhưng làm mất thông tin support của các tập con. CFI vẫn giữ đủ thông tin support để suy ra support của các FI thông thường trong nhiều trường hợp.

1.2 Tính chất Apriori

Một tính chất nền tảng của FIM là tính chất Apriori, còn gọi là tính chất đơn điệu giảm của support. Với hai itemset X và Y thỏa $X \subseteq Y$, ta luôn có:

$$\text{sup}(Y) \leq \text{sup}(X). \quad (11)$$

Điều này đúng vì mọi giao dịch chứa Y chắc chắn cũng chứa X , nhưng không phải giao dịch chứa X nào cũng chứa Y . Do đó:

$$\{T_j \in D \mid Y \subseteq T_j\} \subseteq \{T_j \in D \mid X \subseteq T_j\}. \quad (12)$$

Suy ra số phần tử của tập bên trái không lớn hơn số phần tử của tập bên phải, tức là $\text{sup}(Y) \leq \text{sup}(X)$.

Từ tính chất trên, ta có hai hệ quả quan trọng:

- Nếu một itemset X không phổ biến, tức $\text{sup}(X) < \text{minsup}$, thì mọi tập cha $Y \supset X$ cũng không phổ biến.
- Nếu một itemset Y phổ biến, thì mọi tập con $X \subseteq Y$ cũng phổ biến.

Tính chất này là cơ sở để tĩa không gian tìm kiếm trong hầu hết các thuật toán FIM, từ Apriori đến các thuật toán theo chiều sâu như Eclat, FP-Growth và PrePost.

1.3 Tổng quan thuật toán PrePost

PrePost là thuật toán FIM dựa trên cấu trúc cây và danh sách mã hóa node. Thuật toán được đề xuất nhằm giảm chi phí sinh ứng viên và giảm số lần quét TDB so với các thuật toán dựa trên sinh ứng viên như Apriori. Thay vì biểu diễn itemset bằng danh sách giao dịch chứa itemset như Eclat, PrePost biểu diễn itemset bằng N-list, một cấu trúc được xây dựng từ PPC-tree.

Ý tưởng cốt lõi của PrePost gồm ba bước chính:

1. Quét TDB để tìm các frequent 1-itemset và sắp xếp item theo support giảm dần.
2. Xây dựng PPC-tree từ các giao dịch đã được lọc và sắp xếp. Mỗi node trên cây được gán hai mã: *pre* và *post*.
3. Tạo N-list cho từng item phổ biến. Sau đó khai thác các itemset dài hơn bằng cách kết hợp N-list và kiểm tra quan hệ tổ tiên – hậu duệ giữa các node.

Điểm khác biệt quan trọng của PrePost là quan hệ bao hàm giữa các node trên PPC-tree có thể được kiểm tra bằng cặp mã (*pre*, *post*). Với hai node *u* và *v*, node *u* là tổ tiên của node *v* nếu:

$$pre(u) < pre(v) \quad \text{và} \quad post(u) > post(v). \quad (13)$$

Nhờ đó, khi cần tính support của itemset mở rộng, thuật toán không phải quét lại TDB gốc mà chỉ thao tác trên các N-list đã được xây dựng.

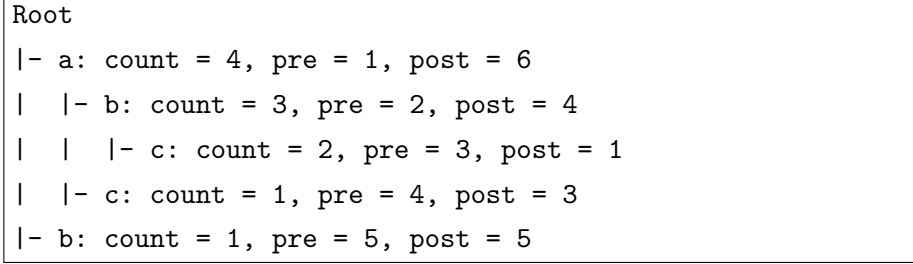
1.4 Cấu trúc dữ liệu PPC-tree

PPC-tree là cây tiền tổ được xây dựng từ TDB sau khi loại bỏ các item không phổ biến và sắp xếp các item còn lại theo một thứ tự toàn cục. Thông thường, thứ tự toàn cục được chọn theo support giảm dần để tăng khả năng chia sẻ tiền tổ giữa các giao dịch.

Mỗi node trên PPC-tree lưu các thông tin chính:

- **item-name**: tên item tương ứng với node.
- **count**: số lượng giao dịch đi qua node đó.
- **children**: danh sách các node con.
- **pre**: chỉ số duyệt trước của node.
- **post**: chỉ số duyệt sau của node.

Ví dụ minh họa dạng tổng quát của một PPC-tree được trình bày trong Hình 1. Trong Chương 2, nhóm sẽ xây dựng chi tiết PPC-tree từ một TDB cụ thể và tính toán đầy đủ các giá trị *pre*, *post*, *count*.



Hình 1: Minh họa cấu trúc PPC-tree với mã *pre* và *post*.

Cặp mã (*pre*, *post*) giúp xác định nhanh quan hệ tổ tiên – hậu duệ. Đây là nền tảng để PrePost chuyển bài toán kiểm tra giao dịch chứa itemset thành bài toán thao tác trên danh sách node.

1.5 Cấu trúc dữ liệu N-list

N-list của một item hoặc itemset là danh sách các bộ ba:

$$(pre, post, count), \quad (14)$$

trong đó mỗi bộ ba tương ứng với một node trên PPC-tree. Với một item đơn *i*, $NList(i)$ chứa tất cả các node mang nhãn *i* trên PPC-tree. Support của item *i* được tính bằng tổng count của các phần tử trong N-list:

$$sup(i) = \sum_{(pre, post, count) \in NList(i)} count. \quad (15)$$

Với itemset có nhiều item, N-list được xây dựng bằng cách kết hợp N-list của các itemset con. Giả sử cần tính N-list của itemset XY từ $NList(X)$ và $NList(Y)$. Thuật toán xét các cặp phần tử (x, y) sao cho node tương ứng với x là tổ tiên của node tương ứng với y trên PPC-tree. Điều kiện tổ tiên được kiểm tra bằng:

$$pre(x) < pre(y) \quad \text{và} \quad post(x) > post(y). \quad (16)$$

Nếu điều kiện đúng, phần tử tương ứng từ Y được dùng để tạo N-list của XY . Support của XY tiếp tục được tính bằng tổng count trong $NList(XY)$:

$$sup(XY) = \sum_{(pre, post, count) \in NList(XY)} count. \quad (17)$$

Nhờ cách biểu diễn này, PrePost tránh phải sinh và kiểm tra ứng viên bằng cách

quét TDB nhiều lần. Phần lớn chi phí tính toán được chuyển sang thao tác trên các N-list, vốn thường nhỏ hơn đáng kể so với TDB ban đầu khi dữ liệu có nhiều tiền tố chung.

1.6 Giải mã thuật toán PrePost

Giải mã tổng quát của thuật toán PrePost được trình bày trong Thuật toán 1. Phần cài đặt cụ thể bằng Julia sẽ được trình bày ở Chương 3.

Listing 1: Giải mã tổng quát của thuật toán PrePost

```

1 Input : Transaction database D, minimum support minsup
2 Output: Set of all frequent itemsets FI
3
4 1. Scan D to compute support of each item.
5 2. Remove infrequent items whose support < minsup.
6 3. Sort frequent items by descending support to obtain global
   order F.
7 4. For each transaction T in D:
8     a. Remove infrequent items from T.
9     b. Sort remaining items according to F.
10    c. Insert the sorted transaction into the PPC-tree.
11 5. Traverse the PPC-tree to assign pre-order and post-order codes
   .
12 6. Construct N-list for each frequent 1-itemset.
13 7. FI = all frequent 1-itemsets.
14 8. Call Mine(prefix = empty, candidate_set = frequent 1-itemsets)
   .
15
16 Procedure Mine(prefix, candidate_set):
17 1. For each item X in candidate_set:
18     a. new_prefix = prefix union {X}
19     b. Output new_prefix as a frequent itemset.
20     c. suffix_set = empty.
21     d. For each item Y after X in candidate_set:
22         i. Compute NList(new_prefix union {Y})
23            from NList(new_prefix) and NList(Y).
24         ii. If support >= minsup, add Y to suffix_set.
25     e. Mine(new_prefix, suffix_set).
```

Trong giải mã trên, thủ tục Mine khai thác không gian tìm kiếm theo chiều sâu. Với mỗi tiền tố hiện tại, thuật toán thử mở rộng tiền tố bằng các item đứng sau theo thứ tự toàn cục. Nếu N-list của itemset mở rộng có support không nhỏ hơn *minsup*, itemset

đó được giữ lại và tiếp tục được mở rộng ở các mức sâu hơn.

1.7 Phân tích độ phức tạp

Độ phức tạp của PrePost phụ thuộc vào số lượng giao dịch, số lượng item, độ dài trung bình của giao dịch, ngưỡng *minsup* và mức độ chia sẻ tiền tố giữa các giao dịch.

Gọi $n = |D|$ là số giao dịch, $m = |I|$ là số item phân biệt và ℓ là độ dài trung bình của giao dịch sau khi loại bỏ item không phổ biến. Việc quét TDB để tính support của các item đơn có chi phí xấp xỉ:

$$O(n \cdot \ell). \quad (18)$$

Sau khi lọc và sắp xếp item, việc xây dựng PPC-tree cũng có chi phí phụ thuộc vào tổng số item xuất hiện trong các giao dịch đã lọc. Nếu dùng cấu trúc ánh xạ phù hợp để tìm node con, chi phí xây dựng cây thường gần với:

$$O(n \cdot \ell). \quad (19)$$

Tuy nhiên, nếu việc tìm node con được cài đặt bằng cách duyệt tuyến tính danh sách con, chi phí có thể tăng theo số lượng node con tại từng mức.

Chi phí khai thác itemset phụ thuộc trực tiếp vào số lượng FI sinh ra và kích thước của các N-list. Trong trường hợp tốt, khi *minsup* cao hoặc dữ liệu có ít item phổ biến, số lượng itemset cần xét nhỏ, N-list ngắn, thuật toán chạy nhanh. Trong trường hợp trung bình, chi phí chủ yếu nằm ở các phép kết hợp N-list. Trong trường hợp xấu nhất, số lượng FI có thể tăng theo hàm mũ theo số item, đặc biệt khi dữ liệu dày đặc và *minsup* thấp. Khi đó, mọi thuật toán FIM hoàn chỉnh đều phải đối mặt với kích thước đầu ra rất lớn.

Độ phức tạp không gian của PrePost gồm hai phần chính:

- Bộ nhớ lưu PPC-tree, phụ thuộc vào số node được tạo ra. Số node tối đa không vượt quá tổng số item trong toàn bộ giao dịch sau lọc.
- Bộ nhớ lưu các N-list, phụ thuộc vào số lượng node liên quan đến từng itemset và số lượng itemset trung gian được giữ trong quá trình khai thác.

Với dữ liệu có nhiều tiền tố chung, PPC-tree có thể nén dữ liệu tốt. Ngược lại, với dữ liệu thưa và ít chia sẻ tiền tố, số node có thể lớn, làm tăng chi phí bộ nhớ.

Các yếu tố ảnh hưởng chính đến hiệu năng của PrePost gồm:

1. **Ngưỡng minsup:** *minsup* càng thấp thì số FI càng lớn, dẫn đến nhiều phép kết hợp N-list hơn.

2. **Độ dày của dữ liệu:** Dữ liệu dày đặc thường tạo nhiều itemset dài, làm tăng kích thước không gian tìm kiếm.
3. **Độ dài giao dịch trung bình:** Giao dịch càng dài thì chi phí xây dựng cây và số tổ hợp item có thể xuất hiện càng lớn.
4. **Mức độ chia sẻ tiền tố:** Chia sẻ tiền tố càng cao thì PPC-tree càng nén tốt, N-list có thể ngắn hơn.
5. **Cách cài đặt phép giao N-list:** Đây là phần ảnh hưởng lớn đến thời gian chạy ở giai đoạn khai thác.

1.8 So sánh lịch sử với các thuật toán FIM khác

Apriori là một trong các thuật toán kinh điển đầu tiên cho FIM. Thuật toán này dựa trên tính chất Apriori để sinh ứng viên theo từng mức. Ưu điểm của Apriori là dễ hiểu và dễ cài đặt, nhưng nhược điểm lớn là phải sinh rất nhiều candidate itemset và phải quét TDB nhiều lần.

FP-Growth cải thiện Apriori bằng cách xây dựng FP-tree để nén TDB và khai thác theo chiến lược chia để trị. Thuật toán này tránh sinh ứng viên trực tiếp, nhưng vẫn cần xây dựng nhiều conditional FP-tree trong quá trình khai thác, gây chi phí đáng kể với một số loại dữ liệu.

Eclat chuyển từ biểu diễn ngang sang biểu diễn dọc bằng tidset. Support của itemset được tính bằng phép giao tidset. Cách tiếp cận này thường hiệu quả khi tidset không quá lớn, nhưng với TDB lớn, các tidset có thể tiêu tốn nhiều bộ nhớ.

PrePost kế thừa ý tưởng khai thác theo chiều sâu và sử dụng biểu diễn mã hóa dựa trên cây. Thuật toán dùng PPC-tree để mã hóa TDB, sau đó dùng N-list để biểu diễn itemset. Nhờ cặp mã *pre* và *post*, quan hệ tổ tiên giữa các node có thể được kiểm tra nhanh mà không cần truy cập lại TDB gốc. Vì vậy, PrePost giải quyết một phần hạn chế của Apriori về sinh ứng viên và hạn chế của FP-Growth về việc xây dựng nhiều cây điều kiện.

Tuy nhiên, PrePost vẫn có hạn chế. Khi dữ liệu thưa, ít chia sẻ tiền tố hoặc *minsup* rất thấp, số lượng N-list trung gian có thể lớn, dẫn đến chi phí bộ nhớ cao. Ngoài ra, hiệu năng của thuật toán phụ thuộc nhiều vào cách cài đặt cấu trúc cây, cách lưu N-list và cách tối ưu phép kết hợp N-list. Các thuật toán sau này như PrePost+ tiếp tục cải tiến PrePost bằng các chiến lược tỉa không gian tìm kiếm, chẳng hạn Children–Parent Equivalence pruning.

1.9 Kết luận chương

Chương này đã trình bày nền tảng lý thuyết của FIM, bao gồm các khái niệm TDB, support, FI, CFI, MFI và tính chất Apriori. Trên cơ sở đó, chương đã giới thiệu thuật toán PrePost, tập trung vào hai cấu trúc dữ liệu chính là PPC-tree và N-list, đồng thời phân tích giả mã, độ phức tạp và vị trí của PrePost trong dòng phát triển các thuật toán FIM. Những nội dung này là nền tảng để Chương 2 trình bày ví dụ minh họa thủ công và Chương 3 triển khai thuật toán bằng ngôn ngữ Julia.

2 Ví dụ minh họa tay thuật toán PrePost

Chương này trình bày hai ví dụ minh họa thuật toán PrePost hoàn toàn bằng tay. Ví dụ thứ nhất là ví dụ cơ sở dùng để mô tả đầy đủ các bước của thuật toán, từ TDB ban đầu, tính support, xây dựng PPC-tree, gán mã *pre* và *post*, tạo N-list, sinh FI và kiểm tra chéo kết quả. Ví dụ thứ hai minh họa tình huống đặc biệt khi PPC-tree chỉ có một nhánh, qua đó cho thấy khả năng nén dữ liệu của cấu trúc cây trong PrePost.

2.1 Ví dụ 1: Cơ sở

2.1.1 Cơ sở dữ liệu giao dịch ban đầu

Xét TDB đồ chơi gồm 5 giao dịch và 5 item phân biệt. Ngưỡng hỗ trợ tối thiểu được chọn là:

$$\text{minsup} = 2. \quad (20)$$

TDB ban đầu được trình bày trong Bảng 2.

Bảng 2: TDB của ví dụ cơ sở

TID	Giao dịch
T_1	$\{1, 2, 3\}$
T_2	$\{1, 3, 4\}$
T_3	$\{2, 3, 5\}$
T_4	$\{1, 2, 3, 5\}$
T_5	$\{2, 3, 4\}$

2.1.2 Bước 1: Tính support của các item đơn

Quét TDB một lần để tính support của từng item. Kết quả được trình bày trong Bảng 3.

Bảng 3: Support của các item đơn trong ví dụ cơ sở

Item	Support
1	3
2	4
3	5
4	2
5	2

Vì tất cả các item đều có support không nhỏ hơn *minsup*, toàn bộ 5 item đều được giữ lại. Thứ tự toàn cục được chọn theo support giảm dần. Khi hai item có cùng support, nhóm dùng thứ tự tăng dần theo mã item để phá vỡ hòa. Do đó thứ tự toàn cục là:

$$3 \succ 2 \succ 1 \succ 4 \succ 5. \quad (21)$$

2.1.3 Bước 2: Sắp xếp lại các giao dịch

Sau khi lọc item không phổ biến và sắp xếp theo thứ tự toàn cục trong Công thức 21, các giao dịch được biến đổi như trong Bảng 4.

Bảng 4: TDB sau khi sắp xếp theo thứ tự toàn cục

TID	Giao dịch ban đầu	Giao dịch sau sắp xếp
T_1	{1, 2, 3}	$\langle 3, 2, 1 \rangle$
T_2	{1, 3, 4}	$\langle 3, 1, 4 \rangle$
T_3	{2, 3, 5}	$\langle 3, 2, 5 \rangle$
T_4	{1, 2, 3, 5}	$\langle 3, 2, 1, 5 \rangle$
T_5	{2, 3, 4}	$\langle 3, 2, 4 \rangle$

2.1.4 Bước 3: Xây dựng PPC-tree

Các giao dịch sau khi sắp xếp được chèn lần lượt vào PPC-tree.

Chèn $T_1 = \langle 3, 2, 1 \rangle$: tạo đường đi $3 \rightarrow 2 \rightarrow 1$, mỗi node có *count* = 1.

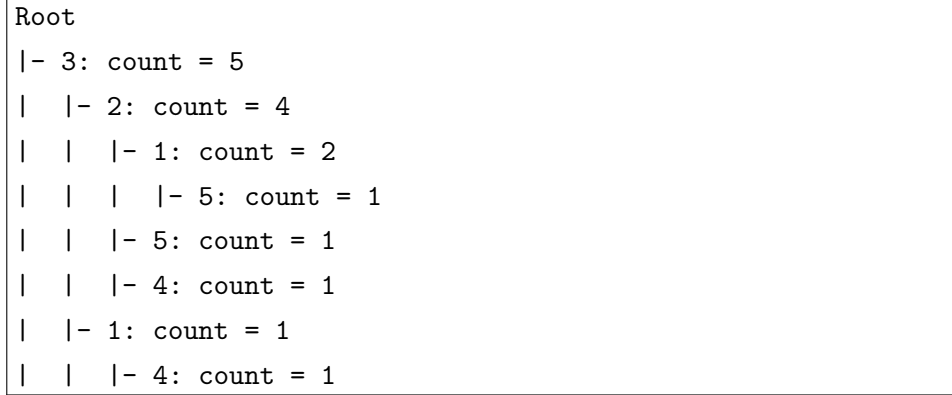
Chèn $T_2 = \langle 3, 1, 4 \rangle$: node 3 đã tồn tại nên tăng *count*(3) lên 2. Sau đó tạo nhánh mới $1 \rightarrow 4$ dưới node 3.

Chèn $T_3 = \langle 3, 2, 5 \rangle$: tăng *count*(3) lên 3, tăng *count*(2) dưới 3 lên 2, sau đó tạo node 5 dưới node 2.

Chèn $T_4 = \langle 3, 2, 1, 5 \rangle$: tăng *count*(3) lên 4, tăng *count*(2) dưới 3 lên 3, tăng *count*(1) dưới 2 lên 2, sau đó tạo node 5 dưới node 1.

Chèn $T_5 = \langle 3, 2, 4 \rangle$: tăng $count(3)$ lên 5, tăng $count(2)$ dưới 3 lên 4, sau đó tạo node 4 dưới node 2.

PPC-tree sau khi chèn toàn bộ giao dịch được minh họa trong Hình 2.



Hình 2: PPC-tree của ví dụ cơ sở trước khi gán mã *pre* và *post*

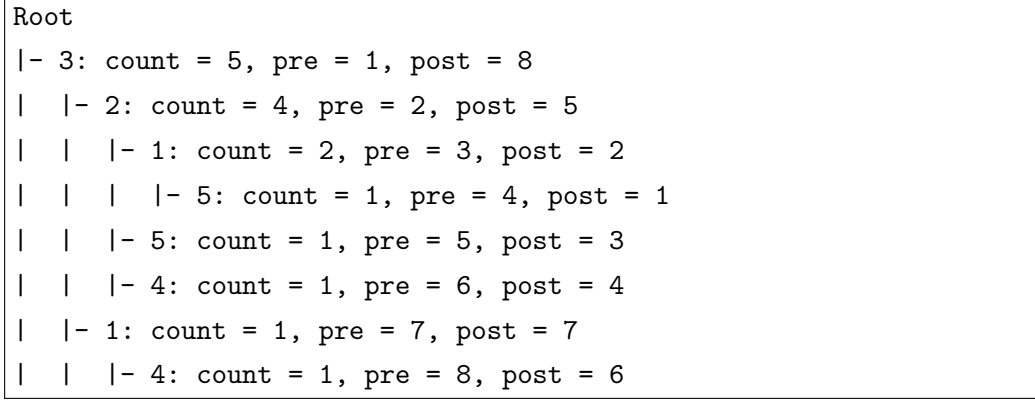
2.1.5 Bước 4: Gán mã *pre* và *post*

Duyệt cây theo thứ tự trước để gán mã *pre*, sau đó dùng thứ tự sau để gán mã *post*. Kết quả được trình bày trong Bảng 5.

Bảng 5: Các node của PPC-tree sau khi gán mã *pre* và *post*

Ký hiệu node	Item	Count	<i>pre</i>	<i>post</i>
n_1	3	5	1	8
n_2	2	4	2	5
n_3	1	2	3	2
n_4	5	1	4	1
n_5	5	1	5	3
n_6	4	1	6	4
n_7	1	1	7	7
n_8	4	1	8	6

PPC-tree có mã *pre* và *post* được minh họa trong Hình 3.

Hình 3: PPC-tree của ví dụ cơ sở sau khi gán mã *pre* và *post*

2.1.6 Bước 5: Tạo N-list cho các item đơn

Từ PPC-tree đã gán mã, N-list của từng item được xây dựng bằng cách gom tất cả các node có cùng item. Kết quả được trình bày trong Bảng 6.

Bảng 6: N-list của các item đơn trong ví dụ cơ sở

Item	N-list	Support
3	$\{(1, 8, 5)\}$	5
2	$\{(2, 5, 4)\}$	4
1	$\{(3, 2, 2), (7, 7, 1)\}$	3
4	$\{(6, 4, 1), (8, 6, 1)\}$	2
5	$\{(4, 1, 1), (5, 3, 1)\}$	2

Support của một item được tính bằng tổng trường *count* trong N-list. Ví dụ:

$$sup(1) = 2 + 1 = 3. \quad (22)$$

2.1.7 Bước 6: Sinh frequent 2-itemset bằng N-list

Để sinh N-list của một 2-itemset, PrePost kiểm tra quan hệ tổ tiên – hậu duệ giữa các node thông qua điều kiện:

$$pre(x) < pre(y) \quad \text{và} \quad post(x) > post(y). \quad (23)$$

Ví dụ, để tính support của itemset $\{2, 3\}$, xét node của item 3 là $(1, 8, 5)$ và node của item 2 là $(2, 5, 4)$. Vì:

$$1 < 2 \quad \text{và} \quad 8 > 5, \quad (24)$$

nên node 3 là tổ tiên của node 2. Do đó:

$$NList(3, 2) = \{(2, 5, 4)\}, \quad (25)$$

và:

$$sup(\{2, 3\}) = 4. \quad (26)$$

Tương tự, các frequent 2-itemset thu được bằng thao tác trên N-list được trình bày trong Bảng 7.

Bảng 7: Các frequent 2-itemset trong ví dụ cơ sở

Itemset	N-list theo thứ tự PPC-tree	Support
$\{1, 2\}$	$NList(2, 1) = \{(3, 2, 2)\}$	2
$\{1, 3\}$	$NList(3, 1) = \{(3, 2, 2), (7, 7, 1)\}$	3
$\{2, 3\}$	$NList(3, 2) = \{(2, 5, 4)\}$	4
$\{2, 5\}$	$NList(2, 5) = \{(4, 1, 1), (5, 3, 1)\}$	2
$\{3, 4\}$	$NList(3, 4) = \{(6, 4, 1), (8, 6, 1)\}$	2
$\{3, 5\}$	$NList(3, 5) = \{(4, 1, 1), (5, 3, 1)\}$	2

Các 2-itemset khác có support nhỏ hơn *minsup* nên bị loại.

2.1.8 Bước 7: Sinh frequent 3-itemset

Từ các frequent 2-itemset, thuật toán tiếp tục mở rộng theo chiều sâu. Các frequent 3-itemset thu được là $\{1, 2, 3\}$ và $\{2, 3, 5\}$.

Với itemset $\{1, 2, 3\}$, theo thứ tự trên PPC-tree ta xét $NList(3, 2)$ và item 1. Node $(3, 2, 2)$ của item 1 là hậu duệ của node $(2, 5, 4)$ của item 2, nên:

$$NList(3, 2, 1) = \{(3, 2, 2)\}. \quad (27)$$

Do đó:

$$sup(\{1, 2, 3\}) = 2. \quad (28)$$

Với itemset $\{2, 3, 5\}$, hai node của item 5 là hậu duệ của node $(2, 5, 4)$ của item 2, nên:

$$NList(3, 2, 5) = \{(4, 1, 1), (5, 3, 1)\}. \quad (29)$$

Do đó:

$$sup(\{2, 3, 5\}) = 1 + 1 = 2. \quad (30)$$

Kết quả frequent 3-itemset được trình bày trong Bảng 8.

Bảng 8: Các frequent 3-itemset trong ví dụ cơ sở		
Itemset	N-list theo thứ tự PPC-tree	Support
$\{1, 2, 3\}$	$NList(3, 2, 1) = \{(3, 2, 2)\}$	2
$\{2, 3, 5\}$	$NList(3, 2, 5) = \{(4, 1, 1), (5, 3, 1)\}$	2

Không có frequent 4-itemset nào vì mọi mở rộng tiếp theo đều có support nhỏ hơn $minsup$.

2.1.9 Tập kết quả cuối cùng

Tập FI cuối cùng của ví dụ cơ sở được trình bày trong Bảng 9.

Bảng 9: Tập FI cuối cùng của ví dụ cơ sở

Itemset	Support
$\{1\}$	3
$\{2\}$	4
$\{3\}$	5
$\{4\}$	2
$\{5\}$	2
$\{1, 2\}$	2
$\{1, 3\}$	3
$\{2, 3\}$	4
$\{2, 5\}$	2
$\{3, 4\}$	2
$\{3, 5\}$	2
$\{1, 2, 3\}$	2
$\{2, 3, 5\}$	2

2.1.10 Kiểm tra chéo bằng liệt kê thủ công

Để kiểm tra chéo kết quả, nhóm liệt kê thủ công support của các itemset có khả năng phổ biến.

Với các item đơn, toàn bộ 5 item đều phổ biến như Bảng 3.

Với các 2-itemset, support thủ công được trình bày trong Bảng 10.

Bảng 10: Kiểm tra chéo support của toàn bộ 2-itemset

Itemset	Các giao dịch chứa itemset	Support
{1, 2}	T_1, T_4	2
{1, 3}	T_1, T_2, T_4	3
{1, 4}	T_2	1
{1, 5}	T_4	1
{2, 3}	T_1, T_3, T_4, T_5	4
{2, 4}	T_5	1
{2, 5}	T_3, T_4	2
{3, 4}	T_2, T_5	2
{3, 5}	T_3, T_4	2
{4, 5}	Không có	0

Với các 3-itemset, support thủ công được trình bày trong Bảng 11.

Bảng 11: Kiểm tra chéo support của toàn bộ 3-itemset

Itemset	Các giao dịch chứa itemset	Support
{1, 2, 3}	T_1, T_4	2
{1, 2, 4}	Không có	0
{1, 2, 5}	T_4	1
{1, 3, 4}	T_2	1
{1, 3, 5}	T_4	1
{1, 4, 5}	Không có	0
{2, 3, 4}	T_5	1
{2, 3, 5}	T_3, T_4	2
{2, 4, 5}	Không có	0
{3, 4, 5}	Không có	0

Với các itemset có kích thước 4 trở lên, không có itemset nào đạt $minsup = 2$. Do đó, kết quả kiểm tra chéo khớp với tập FI sinh ra bởi PrePost trong Bảng 9.

2.2 Ví dụ 2: Tình huống đặc biệt PPC-tree một nhánh

2.2.1 Cơ sở dữ liệu giao dịch ban đầu

Ví dụ thứ hai được thiết kế để tạo ra PPC-tree dạng một nhánh. Đây là tình huống đặc biệt quan trọng vì các giao dịch có mức độ chia sẻ tiền tố rất cao, giúp PPC-tree nén TDB mạnh.

Ngưỡng hỗ trợ tối thiểu vẫn được chọn là:

$$\text{minsup} = 2. \quad (31)$$

TDB của ví dụ đặc biệt được trình bày trong Bảng 12.

Bảng 12: TDB của ví dụ đặc biệt

TID	Giao dịch
T_1	$\{1, 2, 3, 4\}$
T_2	$\{1, 2, 3\}$
T_3	$\{1, 2, 3, 4\}$
T_4	$\{1, 2, 3\}$
T_5	$\{1, 2\}$

2.2.2 Bước 1: Tính support và thứ tự toàn cục

Support của các item đơn được trình bày trong Bảng 13.

Bảng 13: Support của các item đơn trong ví dụ đặc biệt

Item	Support
1	5
2	5
3	4
4	2

Tất cả item đều phổ biến. Thứ tự toàn cục được chọn là:

$$1 \succ 2 \succ 3 \succ 4. \quad (32)$$

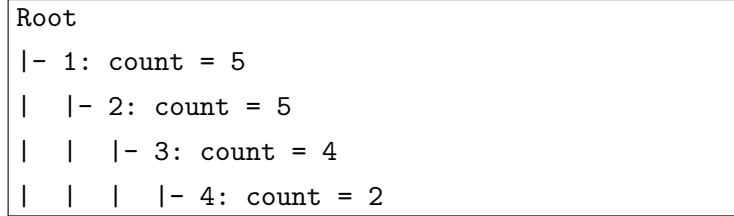
Sau khi sắp xếp, các giao dịch vẫn giữ dạng như Bảng 14.

Bảng 14: TDB sau khi sắp xếp trong ví dụ đặc biệt

TID	Giao dịch sau sắp xếp
T_1	$\langle 1, 2, 3, 4 \rangle$
T_2	$\langle 1, 2, 3 \rangle$
T_3	$\langle 1, 2, 3, 4 \rangle$
T_4	$\langle 1, 2, 3 \rangle$
T_5	$\langle 1, 2 \rangle$

2.2.3 Bước 2: Xây dựng PPC-tree một nhánh

Khi chèn các giao dịch vào PPC-tree, tất cả giao dịch đều chia sẻ cùng một tiền tố $1 \rightarrow 2$. Các giao dịch chứa item 3 tiếp tục đi qua node 3, và các giao dịch chứa item 4 tiếp tục đi qua node 4. Vì vậy, PPC-tree thu được chỉ có một nhánh như Hình 4.



Hình 4: PPC-tree một nhánh trong ví dụ đặc biệt

2.2.4 Bước 3: Gán mã pre và post

Vì cây chỉ có một nhánh, thứ tự *pre* tăng dần từ gốc xuống lá, còn thứ tự *post* tăng dần từ lá lên gần gốc. Kết quả được trình bày trong Bảng 15.

Bảng 15: Mã *pre* và *post* trong PPC-tree một nhánh

Ký hiệu node	Item	Count	<i>pre</i>	<i>post</i>
n_1	1	5	1	4
n_2	2	5	2	3
n_3	3	4	3	2
n_4	4	2	4	1

2.2.5 Bước 4: Tạo N-list

Vì mỗi item chỉ xuất hiện trên đúng một node, N-list của mỗi item chỉ có một bộ ba. Kết quả được trình bày trong Bảng 16.

Bảng 16: N-list của các item đơn trong ví dụ đặc biệt

Item	N-list	Support
1	$\{(1, 4, 5)\}$	5
2	$\{(2, 3, 5)\}$	5
3	$\{(3, 2, 4)\}$	4
4	$\{(4, 1, 2)\}$	2

2.2.6 Bước 5: Khai thác các FI

Trong PPC-tree một nhánh, mọi item đứng trước trên đường đi đều là tổ tiên của các item đứng sau. Vì vậy, mọi tổ hợp item trên nhánh đều là một itemset có thể xét. Support

của itemset chính là count của node sâu nhất trong itemset đó.

Ví dụ, với itemset $\{1, 2, 3, 4\}$, node sâu nhất là node 4 có $count = 2$, do đó:

$$sup(\{1, 2, 3, 4\}) = 2. \quad (33)$$

Vì $sup(\{1, 2, 3, 4\}) = minsup$, itemset này là FI.

Toàn bộ FI của ví dụ đặc biệt được trình bày trong Bảng 17.

Bảng 17: Tập FI cuối cùng của ví dụ đặc biệt

Itemset	Support
$\{1\}$	5
$\{2\}$	5
$\{3\}$	4
$\{4\}$	2
$\{1, 2\}$	5
$\{1, 3\}$	4
$\{1, 4\}$	2
$\{2, 3\}$	4
$\{2, 4\}$	2
$\{3, 4\}$	2
$\{1, 2, 3\}$	4
$\{1, 2, 4\}$	2
$\{1, 3, 4\}$	2
$\{2, 3, 4\}$	2
$\{1, 2, 3, 4\}$	2

2.2.7 Phân tích ý nghĩa của trường hợp đặc biệt

Trường hợp PPC-tree một nhánh là một tình huống đặc biệt quan trọng đối với PrePost. Khi TDB có nhiều giao dịch chia sẻ cùng tiền tố, PPC-tree nén dữ liệu rất tốt vì nhiều giao dịch được biểu diễn bằng cùng một đường đi. Trong ví dụ này, thay vì lưu lại 5 giao dịch riêng biệt, PPC-tree chỉ cần 4 node để biểu diễn toàn bộ TDB.

Ngoài ra, thao tác sinh FI cũng trở nên trực quan hơn. Vì tất cả item nằm trên cùng một nhánh, quan hệ tổ tiên – hậu duệ giữa các node luôn rõ ràng. Với mọi itemset nằm trên nhánh, support được xác định bởi count của node sâu nhất. Điều này giúp giảm đáng kể chi phí kiểm tra quan hệ giữa các node trong N-list.

Tuy nhiên, trường hợp này cũng cho thấy một đặc điểm quan trọng của FIM: khi dữ liệu dày đặc hoặc có nhiều item cùng xuất hiện, số lượng FI có thể tăng nhanh. Trong ví dụ này chỉ có 4 item nhưng đã sinh ra toàn bộ $2^4 - 1 = 15$ FI. Điều này giải thích vì

sao khi *minsup* thấp hoặc dữ liệu dày đặc, thời gian chạy của các thuật toán FIM có thể tăng mạnh do kích thước đầu ra lớn.

2.3 Kết luận chương

Chương này đã trình bày hai ví dụ minh họa tay cho thuật toán PrePost. Ví dụ cơ sở cho thấy đầy đủ quá trình xử lý từ TDB ban đầu đến PPC-tree, N-list và tập FI cuối cùng. Kết quả cũng được kiểm tra chéo bằng cách liệt kê thủ công support của các itemset. Ví dụ thứ hai minh họa trường hợp PPC-tree một nhánh, qua đó làm rõ khả năng nén dữ liệu của PPC-tree và cách PrePost xử lý khi các giao dịch chia sẻ tiền tố cao.

3 Cài đặt thuật toán PrePost

Chương này trình bày quá trình cài đặt thuật toán PrePost từ đầu bằng ngôn ngữ Julia. Mục tiêu của phần cài đặt là hiện thực đầy đủ các bước đã trình bày ở Chương 1 và minh họa bằng tay ở Chương 2: đọc cơ sở dữ liệu giao dịch, tiền xử lý theo ngưỡng *minsup*, xây dựng PPC-tree, gán mã *pre* và *post*, tạo N-list, khai thác toàn bộ frequent itemset và xuất kết quả theo định dạng tương thích với SPMF.

Việc cài đặt được tổ chức theo bốn mức yêu cầu. Level 1 tập trung vào tính đúng đắn của thuật toán cơ bản. Level 2 bổ sung kiểm thử tự động trên nhiều CSDL nhỏ. Level 3 cài đặt một phiên bản tối ưu và đối chiếu với phiên bản cơ bản. Level 4 hoàn thiện phần đọc/ghi file và giao diện dòng lệnh.

3.1 Môi trường và công cụ

Thuật toán được cài đặt bằng Julia, phù hợp với định hướng của đề bài là ưu tiên Julia phiên bản từ 1.9 trở lên. Mã nguồn được tổ chức dưới dạng một project Julia có module chính là `PrePostFIM`. Việc chạy chương trình, kiểm thử và đối chiếu kết quả được thực hiện thông qua các script dòng lệnh trên Windows PowerShell.

Môi trường cài đặt được tóm tắt trong Bảng 18.

Bảng 18: Môi trường và công cụ cài đặt

Thành phần	Mô tả
Ngôn ngữ lập trình	Julia
Module chính	PrePostFIM
Hệ điều hành kiểm thử	Windows, chạy qua PowerShell
Định dạng input	SPMF, mỗi giao dịch là một dòng, các item cách nhau bằng khoảng trắng
Định dạng output	SPMF, dạng <code>itemset #SUP: support</code>
Công cụ kiểm thử	Test của Julia và các script <code>.ps1</code>
Script chính	<code>run_tests.ps1</code> , <code>check_algorithms.ps1</code> , <code>run_optimization_check.ps1</code>

Trong phạm vi chương này, các tập dữ liệu kiểm thử là các CSDL toy có kích thước nhỏ. Các tập này được dùng để kiểm tra tính đúng đắn của từng bước cài đặt và đối chiếu với kết quả mong đợi. Việc đánh giá trên các tập benchmark lớn và so sánh với SPMF được trình bày ở Chương thực nghiệm.

3.2 Tổ chức mã nguồn

Mã nguồn được chia thành nhiều file theo chức năng để giúp dễ kiểm thử và bảo trì. Module **PrePostFIM** đóng vai trò gom các thành phần con và export những hàm cần dùng ở bên ngoài. Cách tổ chức này giúp tách rõ bốn nhóm nhiệm vụ: biểu diễn dữ liệu, tiền xử lý, xây dựng cấu trúc PPC-tree/N-list và khai thác frequent itemset.

Bảng 19 mô tả vai trò của các file chính trong project.

Bảng 19: Tổ chức các file mã nguồn chính

File	Vai trò
PrePostFIM.jl	Module chính, include các file thành phần và export các kiểu dữ liệu, hàm đọc/ghi, hàm khai thác và hàm kiểm tra kết quả.
structures/transaction_db.jl	Định nghĩa kiểu TransactionDB để lưu danh sách giao dịch và các hàm thống kê cơ bản như số giao dịch, số item, độ dài giao dịch trung bình.
structures/ppc_tree.jl	Định nghĩa PPCNode , PPCTree , thao tác chèn giao dịch vào cây, gán mã <i>pre-post</i> và thu thập node.
structures/nlist.jl	Định nghĩa NListEntry , tính support của N-list, tạo N-list cho item đơn và giao hai N-list.
utils/preprocessing.jl	Đếm support item đơn, lọc item không phổ biến, xác định thứ tự toàn cục và sắp xếp/lọc giao dịch.

File	Vai trò
<code>utils/io_spmf.jl</code>	Đọc CSDL theo định dạng SPMF và ghi kết quả frequent itemset theo định dạng SPMF.
<code>utils/metrics.jl</code>	Đọc file output, chuẩn hóa kết quả và so sánh hai tập kết quả.
<code>utils/timer.jl</code>	Đo thời gian chạy của một hàm.
<code>algorithm/output.jl</code>	Chuẩn hóa thứ tự itemset, định dạng một dòng kết quả output.
<code>algorithm/mining.jl</code>	Cài đặt thủ tục khai thác đệ quy từ các N-list ứng viên.
<code>algorithm/prepost.jl</code>	Xây dựng pipeline chính của thuật toán, gồm <code>prepost_basic</code> , <code>prepost_optimized</code> và <code>prepost</code> .
<code>src/cli.jl</code>	Giao diện dòng lệnh, nhận tham số input, minsup và output.

Luồng include trong module chính được tổ chức theo thứ tự phụ thuộc: trước hết là các cấu trúc dữ liệu, sau đó là các tiện ích tiền xử lý/đọc ghi, cuối cùng là các file hiện thực thuật toán. Điều này giúp các hàm trong tầng thuật toán có thể sử dụng trực tiếp các kiểu dữ liệu và hàm tiện ích đã được định nghĩa trước.

3.3 Biểu diễn dữ liệu và cấu trúc chính

3.3.1 Cơ sở dữ liệu giao dịch

CSDL giao dịch được biểu diễn bằng kiểu `TransactionDB`. Mỗi giao dịch là một vector các số nguyên, tương ứng với các item trong giao dịch. Toàn bộ CSDL là một vector các giao dịch:

Listing 2: Biểu diễn CSDL giao dịch

```

1 struct TransactionDB
2     transactions::Vector{Vector{Int}}
3 end

```

Cách biểu diễn này đơn giản và phù hợp với định dạng SPMF, trong đó mỗi dòng input là một giao dịch và các item được mã hóa bằng số nguyên. Trước khi xây dựng PPC-tree, mỗi giao dịch được lọc bỏ các item không phổ biến, loại bỏ item lặp trong cùng giao dịch và sắp xếp theo thứ tự toàn cục.

3.3.2 PPC-tree

PPC-tree được cài đặt bằng hai kiểu dữ liệu chính là `PPCNode` và `PPCTree`. Mỗi node lưu item, số đếm, con trỏ đến cha, danh sách node con, mã *pre* và mã *post*:

Listing 3: Cấu trúc node trong PPC-tree

```

1 mutable struct PPCNode
2     item::Union{Int,Nothing}
3     count::Int
4     parent::Union{PPCNode,Nothing}
5     children::Vector{PPCNode}
6     pre::Int
7     post::Int
8 end

```

Node gốc có `item = nothing`. Khi chèn một giao dịch đã được sắp xếp, thuật toán đi từ gốc xuống theo thứ tự item trong giao dịch. Nếu node con mang item tương ứng đã tồn tại, trường `count` được tăng lên. Ngược lại, một node mới được tạo và thêm vào danh sách con.

Sau khi toàn bộ giao dịch được chèn, cây được duyệt để gán mã `pre` và `post`. Mã `pre` được gán khi bắt đầu thăm node, còn mã `post` được gán sau khi đã thăm toàn bộ các node con. Hai mã này là cơ sở để kiểm tra quan hệ tổ tiên – hậu duệ giữa hai node.

3.3.3 N-list

Một phần tử của N-list được biểu diễn bằng bộ ba $(pre, post, count)$:

Listing 4: Biểu diễn phần tử N-list

```

1 struct NListEntry
2     pre::Int
3     post::Int
4     count::Int
5 end

```

Support của một N-list được tính bằng tổng trường `count` của tất cả phần tử trong N-list:

$$sup(X) = \sum_{e \in NList(X)} e.count. \quad (34)$$

Để tạo N-list cho item đơn, chương trình duyệt toàn bộ node của PPC-tree, gom các node có cùng item và sắp xếp các entry theo mã `pre`. Để sinh N-list cho itemset mở rộng, chương trình kiểm tra quan hệ tổ tiên – hậu duệ giữa N-list của prefix và N-list của item mở rộng. Điều kiện được dùng là:

$$x.pre < y.pre \quad \text{và} \quad x.post > y.post. \quad (35)$$

Nếu điều kiện trên đúng, node x là tổ tiên của node y , và entry của y được giữ lại

trong N-list của itemset mới.

3.4 Quy trình cài đặt thuật toán PrePost

Pipeline chính của thuật toán được cài đặt trong hàm tạo danh sách ứng viên ban đầu và hai hàm khai thác `prepost_basic`, `prepost_optimized`. Các bước xử lý được tóm tắt trong Hình 5.

```
Input SPMF
|
|- Đọc TransactionDB
|- Đếm support item đơn
|- Lọc item có support < minsup
|- Sắp xếp item theo support giảm dần, phá hòa theo mã item tăng dần
|- Lọc và sắp xếp lại từng giao dịch
|- Xây dựng PPC-tree
|- Gán mã pre và post
|- Tạo N-list cho item đơn
|- Khai thác frequent itemset bằng giao N-list
|- Chuẩn hóa và ghi output SPMF
```

Hình 5: Pipeline cài đặt thuật toán PrePost

Các bước tiền xử lý được cài đặt như sau:

1. Quét CSDL để đếm support của từng item đơn. Nếu một item xuất hiện nhiều lần trong cùng một giao dịch thì chỉ được tính một lần.
2. Lọc bỏ các item có support nhỏ hơn *minsup*.
3. Xác định thứ tự toàn cục. Item có support cao hơn đứng trước. Nếu hai item có cùng support, item có mã nhỏ hơn đứng trước.
4. Với từng giao dịch, loại bỏ item không phổ biến, loại bỏ item trùng lặp và sắp xếp theo thứ tự toàn cục.

Sau bước tiền xử lý, các giao dịch được chèn vào PPC-tree. Từ PPC-tree đã gán mã *pre-post*, chương trình xây dựng N-list của các item đơn và khai thác frequent itemset theo chiều sâu.

3.5 Phiên bản cơ bản và phiên bản tối ưu

Project cài đặt hai phiên bản thuật toán:

- **prepost_basic**: phiên bản cơ bản, sinh các N-list ứng viên trước rồi mới lọc các ứng viên có support nhỏ hơn *minsup*.
- **prepost_optimized**: phiên bản tối ưu, kiểm tra support ngay sau khi giao N-list và chỉ đưa những ứng viên đủ *minsup* vào danh sách mở rộng tiếp theo.

Sự khác biệt chính nằm ở vị trí tủa ứng viên. Trong phiên bản cơ bản, các ứng viên được sinh ra đầy đủ ở mỗi mức rồi mới được lọc. Trong phiên bản tối ưu, thao tác lọc được thực hiện sớm hơn, ngay sau khi N-list mới được tạo. Điều này giúp giảm số lượng ứng viên truyền xuống các lời gọi đệ quy tiếp theo.

Về mặt kết quả, hai phiên bản phải sinh ra cùng một tập frequent itemset. Do đó, phiên bản cơ bản được dùng như một đối chứng nội bộ để kiểm tra phiên bản tối ưu. Trong mã nguồn, hàm **prepost** mặc định gọi phiên bản tối ưu:

Listing 5: Hàm PrePost mặc định

```

1 prepost(db::TransactionDB, minsup::Int)::Vector{Tuple{Vector{Int
  },Int}} =
2   prepost_optimized(db, minsup)
```

Thủ tục khai thác đệ quy của phiên bản tối ưu có thể mô tả bằng giả mã trong Thuật toán 6.

```

Mine(prefix, suffixCandidates, minsup)
for each candidate (item, nlist) in suffixCandidates do
  supp ← support(nlist)
  if supp < minsup then continue
  newPrefix ← prefix ∪ {item}
  output (newPrefix, supp)
  nextCandidates ← ∅
  for each later candidate (nextItem, nextNList) do
    joined ← join_nlists(nlist, nextNList)
    if support(joined) ≥ minsup then
      add (nextItem, joined) to nextCandidates
  if nextCandidates is not empty then
    Mine(newPrefix, nextCandidates, minsup)
```

Hình 6: Giả mã khai thác đệ quy bằng N-list trong phiên bản tối ưu

Kỹ thuật tủa sớm này không làm thay đổi kết quả vì mọi itemset có support nhỏ hơn *minsup* không thể là frequent itemset. Đồng thời, theo tính chất anti-monotone của support, việc mở rộng một itemset không phổ biến cũng không thể tạo ra itemset phổ biến.

3.6 Xử lý đầu vào, đầu ra và giao diện dòng lệnh

Chương trình hỗ trợ đọc input theo định dạng SPMF. Mỗi dòng trong file input là một giao dịch, các item được phân tách bằng khoảng trắng. Các phần sau ký tự # được xem là chú thích và bị bỏ qua khi đọc dữ liệu. Các dòng rỗng cũng được bỏ qua.

Ví dụ một file input hợp lệ:

Listing 6: Ví dụ input theo định dạng SPMF

```
1 1 2 3
2 1 3 4
3 2 3 5
4 1 2 3 5
5 2 3 4
```

Kết quả output được ghi theo định dạng:

Listing 7: Ví dụ output theo định dạng SPMF

```
1 1 #SUP: 3
2 2 #SUP: 4
3 3 #SUP: 5
4 1 2 #SUP: 2
```

Trước khi ghi output, các kết quả được chuẩn hóa. Mỗi itemset được sắp xếp tăng dần theo mã item. Toàn bộ danh sách kết quả được sắp xếp theo kích thước itemset tăng dần; nếu hai itemset có cùng kích thước thì sắp xếp theo thứ tự từ điển. Việc chuẩn hóa này giúp so sánh kết quả ổn định và không phụ thuộc vào thứ tự sinh ra trong quá trình đệ quy.

Giao diện dòng lệnh được cài đặt trong file `src/cli.jl`. Chương trình nhận ba tham số bắt buộc: đường dẫn input, ngưỡng *minsup* và đường dẫn output. Cú pháp chạy như sau:

Listing 8: Cú pháp chạy chương trình

```
1 julia --project=. src/cli.jl --input <path> --minsup <integer> --
  output <path>
```

Ví dụ chạy trên CSDL toy cơ sở:

Listing 9: Ví dụ chạy chương trình trên CSDL toy

```
1 julia --project=. src/cli.jl \
2   --input data/toy/example_basic.txt \
3   --minsup 2 \
4   --output outputs/toy/output_basic.out
```


Sau khi chạy, chương trình in ra thông tin input, output, *minsup*, số lượng frequent itemset và thời gian chạy. Điều này giúp thuận tiện cho việc ghi log thực nghiệm.

3.7 Kiểm thử Level 1 và Level 2

3.7.1 Mục tiêu kiểm thử

Mục tiêu kiểm thử là đảm bảo từng thành phần riêng lẻ và toàn bộ pipeline PrePost hoạt động đúng. Các test bao gồm:

- kiểm tra đọc file SPMF;
- kiểm tra đếm support item đơn;
- kiểm tra lọc item không phổ biến và sắp xếp giao dịch;
- kiểm tra xây dựng PPC-tree;
- kiểm tra gán mã *pre* và *post*;
- kiểm tra tạo và giao N-list;
- kiểm tra kết quả frequent itemset cuối cùng;
- kiểm tra chuẩn hóa, ghi và đọc output.

Kết quả chạy unit test mới nhất được ghi nhận như sau:

Listing 10: Kết quả chạy unit test

```

1 Running Julia tests...
2 Starting PrePostFIM test suite...
3 Test Summary: | Pass  Total  Time
4 PrePostFIM    |  103    103   3.9s
5 Finished PrePostFIM test suite.
```

Như vậy, toàn bộ 103 test đều vượt qua.

3.7.2 Bộ dữ liệu kiểm thử Level 2

Để đáp ứng yêu cầu Level 2, nhóm sử dụng năm CSDL toy khác nhau. Hai CSDL đầu là hai ví dụ minh họa tay ở Chương 2. Ba CSDL còn lại được thiết kế để kiểm tra các tình huống khác nhau: dữ liệu thừa, dữ liệu có item không phổ biến và dữ liệu có giao dịch trùng với itemset dài.

Bảng 20 trình bày bộ dữ liệu kiểm thử Level 2.

Bảng 20: Năm CSDL toy dùng cho kiểm thử Level 2

Dataset	Mục đích kiểm thử	<i>minsup</i>	Số FI
<code>example_basic.txt</code>	Ví dụ cơ sở ở Chương 2	2	13
<code>example_special_single_path.txt</code>	Tạo ra tập PPC-tree một nhánh	2	15
<code>example_sparse.txt</code>	Dữ liệu thưa, ít itemset dài	2	5
<code>example_with_infrequent_items.txt</code>	Các itemset không phổ biến cần bị loại	2	6
<code>example_duplicates_long.txt</code>	Có giao dịch trùng và itemset dài	2	15

Mỗi dataset có một file expected output tương ứng trong thư mục `data/toy/expected`. Script `check_algorithms.ps1` chạy chương trình trên năm dataset, chuẩn hóa output rồi so sánh với expected output.

Kết quả kiểm tra thuật toán được tóm tắt trong Bảng 21.

Bảng 21: Kết quả kiểm tra thuật toán trên năm CSDL toy

Dataset	<i>minsup</i>	Số FI sinh ra	Kết quả so sánh
Basic	2	13	Passed
Special single-path	2	15	Passed
Sparse	2	5	Passed
Infrequent-items	2	6	Passed
Duplicates-long	2	15	Passed

Log thực thi của script kiểm tra thuật toán cho thấy cả năm dataset đều khớp expected output:

Listing 11: Kết quả chạy script kiểm tra thuật toán

```

1 [4/5] Comparing results...
2 [PASSED] Basic dataset output matches expected.
3 [PASSED] Special dataset output matches expected.
4 [PASSED] Sparse dataset output matches expected.
5 [PASSED] Infrequent-items dataset output matches expected.
6 [PASSED] Duplicates-long dataset output matches expected.
7
8 =====
9 Algorithm check PASSED
10 =====

```

Tỉ lệ đúng trên bộ kiểm thử Level 2 là:

$$Accuracy = \frac{5}{5} \times 100\% = 100\%. \quad (36)$$

Kết quả này cho thấy chương trình sinh đúng số lượng frequent itemset và đúng support tương ứng trên toàn bộ năm CSDL toy.

3.8 Kiểm tra Level 3: so sánh phiên bản cơ bản và tối ưu

Level 3 yêu cầu áp dụng ít nhất một kỹ thuật tối ưu phù hợp với thuật toán và đo lường mức cải thiện so với bản cơ bản. Trong project này, kỹ thuật được dùng là tĩa sớm ứng viên trong quá trình giao N-list. Cụ thể, sau khi sinh N-list cho một ứng viên mở rộng, chương trình tính support ngay. Nếu support nhỏ hơn *minsup*, ứng viên đó không được đưa vào danh sách mở rộng tiếp theo.

Script `run_optimization_check.ps1` so sánh hai hàm `prepost_basic` và `prepost_optimized` trên năm CSDL toy. Với mỗi dataset, script ghi nhận số lượng itemset, biến `match`, thời gian chạy của bản cơ bản, thời gian chạy của bản tối ưu và `speedup`.

Kết quả được trình bày trong Bảng 22.

Bảng 22: Kết quả so sánh phiên bản cơ bản và phiên bản tối ưu

Dataset	<i>minsup</i>	Số FI	Match	Basic (s)	Optimized (s)
<code>toy_basic</code>	2	13	true	0.189	0.000
<code>toy_special_single_path</code>	2	15	true	0.081	0.000
<code>toy_sparse</code>	2	5	true	0.000	0.000
<code>toy_with_infrequent_items</code>	2	6	true	0.000	0.000
<code>toy_duplicates_long</code>	2	15	true	0.000	0.000

Toàn bộ năm dataset đều có `match=true`, nghĩa là phiên bản tối ưu sinh ra cùng tập frequent itemset với phiên bản cơ bản. Đây là điều kiện quan trọng nhất khi đánh giá một tối ưu hóa thuật toán: tối ưu không được làm thay đổi kết quả.

Một số giá trị thời gian được làm tròn về 0 giây do các CSDL toy rất nhỏ và thời gian chạy thấp hơn độ phân giải đo thời gian trong log. Vì vậy, các giá trị `speedup` như `Inf` chỉ nên xem là tín hiệu tham khảo, không nên diễn giải là tốc độ tăng vô hạn. Đánh giá hiệu năng đáng tin cậy hơn cần được thực hiện trên các tập benchmark lớn ở Chương thực nghiệm. Trong phạm vi Chương 3, kết quả quan trọng là bản tối ưu khớp hoàn toàn với bản cơ bản trên toàn bộ kiểm thử.

3.9 Đáp ứng các mức yêu cầu cài đặt

Bảng 23 tổng hợp mức độ đáp ứng các yêu cầu cài đặt.

Bảng 23: Tổng hợp mức độ đáp ứng các yêu cầu cài đặt

Mức	Yêu cầu	Kết quả thực hiện
Level 1	Cài đặt đúng thuật toán, xuất toàn bộ frequent itemset và support tương ứng.	Đã cài đặt pipeline PrePost đầy đủ từ đọc dữ liệu, tiền xử lý, PPC-tree, N-list đến khai thác FI. Kết quả toy cơ sở sinh 13 FI và toy single-path sinh 15 FI, khớp expected.
Level 2	Kiểm thử tự động trên ít nhất 5 CSDL khác nhau, bao gồm CSDL từ ví dụ tay.	Đã kiểm thử trên 5 CSDL toy: basic, special single-path, sparse, infrequent-items và duplicates-long. Unit test đạt 103/103; algorithm check pass 5/5 dataset.
Level 3	Tối ưu hóa bộ nhớ/tốc độ và đo mức cải thiện so với bản cơ bản.	Đã cài đặt bản <code>prepost_optimized</code> với tải sớm ứng viên sau khi giao N-list. So sánh với <code>prepost_basic</code> cho kết quả <code>match=true</code> trên 5/5 dataset.
Level 4	Hỗ trợ đọc input/output định dạng SPMF và tham số dòng lệnh cho <i>minsup</i> , input, output.	Đã có <code>read_spmf</code> , <code>write_spmf_output</code> và CLI nhận <code>-input</code> , <code>-minsup</code> , <code>-output</code> .

3.10 Nhận xét về cài đặt

Cài đặt hiện tại có một số ưu điểm chính. Thứ nhất, mã nguồn được chia module rõ ràng, giúp kiểm thử độc lập từng thành phần. Thứ hai, output được chuẩn hóa trước khi ghi và so sánh, nhờ đó việc kiểm tra kết quả không bị ảnh hưởng bởi thứ tự sinh itemset trong quá trình đệ quy. Thứ ba, project duy trì đồng thời phiên bản cơ bản và phiên bản tối ưu, cho phép kiểm tra chéo nội bộ trước khi đánh giá trên dữ liệu lớn.

Tuy nhiên, một số điểm cần lưu ý vẫn còn tồn tại. Bộ kiểm thử ở Chương 3 chủ yếu sử dụng dữ liệu toy nên chưa phản ánh đầy đủ hiệu năng trên dữ liệu lớn. Ngoài ra,

thời gian chạy của các dataset nhỏ rất thấp, khiến việc đo speedup chưa ổn định. Vì vậy, các kết luận về tốc độ ở Chương này chỉ có giá trị kiểm tra ban đầu. Phần đánh giá hiệu năng đầy đủ cần dựa trên các tập benchmark lớn, nhiều mức *minsup* khác nhau và so sánh với SPMF trong Chương thực nghiệm.

3.11 Kết luận chương

Chương này đã trình bày quá trình cài đặt thuật toán PrePost bằng Julia. Chương trình được xây dựng từ đầu, bao gồm các thành phần đọc dữ liệu, tiền xử lý, xây dựng PPC-tree, gán mã *pre-post*, tạo N-list, khai thác frequent itemset và xuất kết quả theo định dạng SPMF. Các yêu cầu từ Level 1 đến Level 4 đều đã được đáp ứng ở mức cài đặt.

Kết quả kiểm thử cho thấy 103/103 unit test đều vượt qua. Script kiểm tra thuật toán chạy thành công trên năm CSDL toy khác nhau và toàn bộ output đều khớp expected. Phiên bản tối ưu cũng cho kết quả trùng khớp với phiên bản cơ bản trên năm dataset. Những kết quả này cung cấp cơ sở để chuyển sang Chương thực nghiệm, nơi thuật toán sẽ được đánh giá trên các tập dữ liệu benchmark lớn và so sánh với SPMF.

4 Thực nghiệm và đánh giá

5 Ứng dụng thực tế

Tài liệu tham khảo

- [1] R. Agrawal and R. Srikant, "Fast Algorithms for Mining Association Rules," in *Proceedings of the 20th International Conference on Very Large Data Bases* Morgan Kaufmann, 1994, pages 487–499.
- [2] J. Han, J. Pei and Y. Yin, "Mining Frequent Patterns without Candidate Generation," in *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data* ACM, 2000, pages 1–12. DOI: [10.1145/342009.335372](https://doi.org/10.1145/342009.335372)
- [3] Z.-H. Deng, Z.-H. Wang and J.-J. Jiang, "A New Algorithm for Fast Mining Frequent Itemsets Using N-lists," *Science China Information Sciences*, **jourvol** 55, **number** 9, pages 2008–2030, 2012. DOI: [10.1007/s11432-012-4638-z](https://doi.org/10.1007/s11432-012-4638-z)
- [4] Z.-H. Deng and S.-H. Lv, "PrePost+: An Efficient N-lists-based Algorithm for Mining Frequent Itemsets via Children-Parent Equivalence Pruning," *Expert Systems with Applications*, **jourvol** 42, **number** 13, pages 5424–5432, 2015. DOI: [10.1016/j.eswa.2015.03.004](https://doi.org/10.1016/j.eswa.2015.03.004)
- [5] M. J. Zaki, "Scalable Algorithms for Association Mining," in *IEEE Transactions on Knowledge and Data Engineering* **volume** 12, 2000, pages 372–390. DOI: [10.1109/69.846291](https://doi.org/10.1109/69.846291)
- [6] P. Fournier-Viger, A. Gomariz, T. Gueniche, A. Soltani, C.-W. Wu and V. S. Tseng, "SPMF: A Java Open-Source Pattern Mining Library," *Journal of Machine Learning Research*, **jourvol** 15, pages 3389–3393, 2014. url: <https://www.jmlr.org/papers/v15/fournierviger14a.html>
- [7] P. Fournier-Viger, *SPMF: An Open-Source Data Mining Library*, <https://www.philippe-fournier-viger.com/spmf/>, Accessed: 2026-01-01, 2026.
- [8] Frequent Itemset Mining Dataset Repository, *FIMI Repository*, <http://fimi.uantwerpen.be/data/>, Accessed: 2026-01-01, 2026.
- [9] J. Han, M. Kamber and J. Pei, *Data Mining: Concepts and Techniques*, 3 edition. Morgan Kaufmann, 2011.

A Phụ lục